M/M/c/k Queue Simulation

Simulates a multi-server queueing system using discrete-event simulation.

Queue Model (Kendall Notation)

- **M/M/c/k** = Markovian arrivals -> Markovian service -> c servers -> max k customers in system

What It Does

Simulates customers arriving, getting served by available servers, or waiting in queue. Rejects arrivals when system is full (queue + servers >= k).

Inputs

- `k` - system capacity (max customers in system), must be >= 1
- `c` - number of parallel servers, must be >= 1
- `lambda` (lambda) - arrival rate (customers/time unit), must be > 0
- `mu` (mu) - service rate (customers/time unit), must be > 0
- simulation runtime

Input Validation

The program validates all inputs and re-prompts on invalid values:

- `k` and `c` must be positive integers
- `lambda` and `mu` must be positive numbers (floats)

Output

- Event log showing arrivals, service starts, departures
- Metrics:
  - `ts` - average time in system
  - `tq` - average time in queue
  - `ns` - average number in system (Little's Law)
  - `nq` - average number in queue (Little's Law)

Functions

Function	Purpose
`get_inputs()`	Prompts user for k, c, lambda, mu, simulation time
`run_simulation()`	Main simulation loop - tracks clock, queue, servers, events
`calc_metrics()`	Computes ts, tq, ns, nq from collected timing data
`main()`	Orchestrates input -> simulation -> output

How Calculations Work

Time Progression

- Event-driven: clock jumps to next arrival OR next departure (whichever is sooner)
- `next_arrival = clock + random.expovariate(lam)` generates next arrival time
- `svc_time = random.expovariate(mu)` generates service duration

Arrival Flow (run_simulation)

1. Customer arrives -> check total in system (queue + busy servers)
2. If >= k -> **rejected**
3. Else if free server -> start service immediately, set service end time
4. Else -> add to **queue**

Departure Flow (run_simulation)

1. Server finishes -> customer departs, server becomes free
2. If queue not empty -> pop first customer, start service immediately

Metrics Calculation (calc_metrics)

```
ts = mean(completion_time - arrival_time)     # time in system
tq = mean(service_start_time - arrival_time)   # time in queue
lam_eff = completed_customers / total_time    # effective arrival rate
ns = lam_eff * ts                              # Little's Law
nq = lam_eff * tq                              # Little's Law
```